Name: Y.Veerendar Reddy **Reg.No:-** 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT – 14

Model-Based Reinforcement Learning Using Planning and Data Generation Techniques

Aim

To implement a simple Model-Based Reinforcement Learning approach using an environment model and planning, and analyze the learning performance of the agent.

Procedure

1. Define the states, actions, rewards, and transition model.
2. Initialize the value function.
3. Generate experience using the environment model.
4. Use the generated data for planning.
5. Update the state-action values using the Bellman equation.
6. Select the best action based on the learned Q-values.
7. Repeat the planning process for several iterations.
8. Visualize the learned Q-values and policy.

Code:

```
# ============================================================
# EXPERIMENT 14
# MODEL-BASED REINFORCEMENT LEARNING
# Planning and Data Generation
# ============================================================

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# ------------------------------------------------------------
# States and Actions
```

```
# ------------------------------------------------------------

states = ["Start", "Middle", "Goal"]
actions = ["Move", "Stay"]

n_states = len(states)
n_actions = len(actions)

# ------------------------------------------------------------
# Transition Model
# ------------------------------------------------------------

def transition(state, action):

    if state == 0:
        return 1 if action == 0 else 0

    if state == 1:
        return 2 if action == 0 else 1

    return 2



# ------------------------------------------------------------
# Reward Model
# ------------------------------------------------------------

def reward(state, action):

    if state == 0 and action == 0:
        return 1

    if state == 1 and action == 0:
        return 10
```

```
    if action == 1:
        return -1

    return 0


# ------------------------------------------------------------
# Initialize Q-table
# ------------------------------------------------------------

Q = np.zeros((n_states, n_actions))

gamma = 0.9
planning_steps = 50

history = []

# ------------------------------------------------------------
# Model-Based Planning
# ------------------------------------------------------------

for step in range(planning_steps):

    for state in range(n_states):

        for action in range(n_actions):

            next_state = transition(state, action)
            r = reward(state, action)

            # Bellman optimality update
            Q[state, action] = (
                r + gamma * np.max(Q[next_state])
            )
```

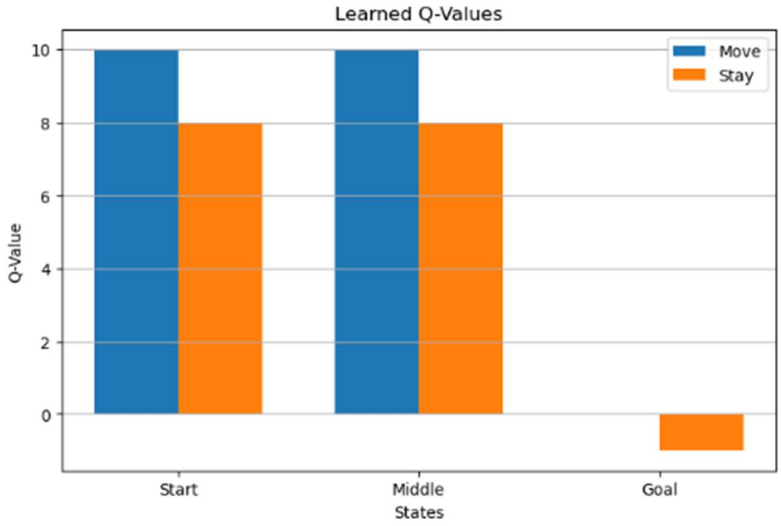
```
    history.append(Q.copy())

# ------------------------------------------------------------
# Best Policy
# ------------------------------------------------------------

policy = np.argmax(Q, axis=1)

# ------------------------------------------------------------
# Display Results
# ------------------------------------------------------------

print("=" * 50)
print("MODEL-BASED REINFORCEMENT LEARNING")
print("=" * 50)

print("\nQ-Table:")
print(Q)

print("\nOptimal Policy:")

for i, state in enumerate(states):

    print(
        f"{state} -> "
        f"{actions[policy[i]]}"
    )

# ------------------------------------------------------------
# Visualization 1: Q-Values
# ------------------------------------------------------------

plt.figure(figsize=(8, 5))

x = np.arange(n_states)
```

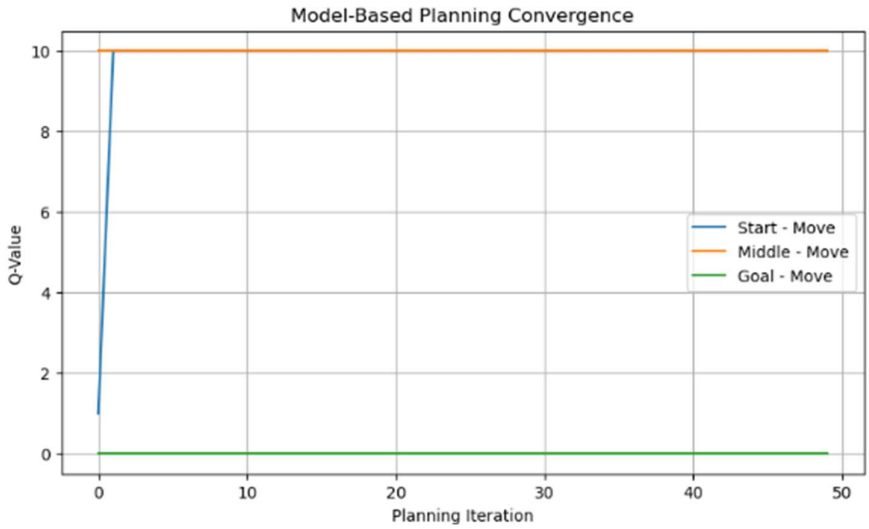
```
width = 0.35

plt.bar(
    x - width / 2,
    Q[:, 0],
    width,
    label="Move"
)

plt.bar(
    x + width / 2,
    Q[:, 1],
    width,
    label="Stay"
)

plt.xticks(x, states)

plt.xlabel("States")
plt.ylabel("Q-Value")

plt.title("Learned Q-Values")

plt.legend()
plt.grid(axis="y")

plt.show()

# ------------------------------------------------------------
# Visualization 2: Planning Convergence
# ------------------------------------------------------------

history = np.array(history)

plt.figure(figsize=(9, 5))
```

```
for state in range(n_states):

    plt.plot(
        history[:, state, 0],
        label=f"{states[state]} - Move"
    )

plt.title("Model-Based Planning Convergence")

plt.xlabel("Planning Iteration")
plt.ylabel("Q-Value")

plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT:-

```
================================================
MODEL-BASED REINFORCEMENT LEARNING
================================================

Q-Table:
[[10.  8.]
 [10.  8.]
 [ 0. -1.]]

Optimal Policy:
Start -> Move
Middle -> Move
Goal -> Move
```





Visualization

The program generates two graphs:

1. Learned Q-Values
   Displays the Q-values of the Move and Stay actions for each state.

2. Planning Convergence
   Shows how the Q-values change during repeated planning iterations.

Summary

A simple Model-Based Reinforcement Learning system was implemented using a known transition and reward model. The agent generated information from the model and used Bellman updates for planning and determining the best policy.

Result

Thus, Model-Based Reinforcement Learning using planning and data generation techniques was successfully implemented, and the learned Q-values and planning convergence were visualized.